🔍 Diff Staged / Unstaged Changes

(Ver diferenças no código)

🧠 Ideia central

O `git diff` mostra **exatamente o que mudou no seu código**

Não é “mais ou menos” — é linha por linha.

📦 Antes de tudo: entenda os dois estados

Você já viu isso, mas aqui fica crucial:

```
Working Directory (não preparado)
Staging Area (preparado com git add)
```

🔍 1. `git diff` — mudanças NÃO preparadas

📌 Comando:

```
git diff
```

🧠 O que ele mostra?

👉 Diferença entre:

- o arquivo atual
- e o último estado salvo (ou staged)

🔧 Exemplo

Arquivo original:

```
preço = 10
```

Você altera:

```
preço = 20
```

Rodando:

```
git diff
```

📄 Saída:

```
- preço = 10
+ preço = 20
```

🧠 Tradução:

- - → linha removida
- + → linha adicionada

🎯 Quando usar?

👉 Antes de fazer `git add`

“O que eu realmente mudei?”

📦 2. `git diff --staged` — mudanças preparadas

📌 Comando:

```
git diff --staged
```

🧠 O que ele mostra?

👉 Diferença entre:

- staging area
- último commit

🔧 Fluxo exemplo

```
git add app.txt
git diff --staged
```

💡 Uso real

👉 Verificar se você adicionou as mudanças certas antes do commit

💧 3. `git diff HEAD` — tudo que mudou

📌 Comando:

```
git diff HEAD
```

🧠 O que mostra?

👉 Tudo que mudou desde o último commit:

- staged
- unstaged

🎯 Quando usar?

👉 Visão geral completa

🧬 4. `git show` — ver mudanças de um commit

📌 Comando:

```
git show <commit>
```

🔧 Exemplo

```
git show HEAD
```

🧠 O que mostra?

- código alterado
- autor
- mensagem

🔍 5. Comparando commits

📌 Comando:

```
git diff commit1 commit2
```

🧠 Uso

👉 ver evolução entre versões

🧠 Visual completo

```
Sem add:
git diff

Com add:
git diff --staged

Tudo:
```

```
git diff HEAD
```

🔥 Exemplo completo (vida real)

```
echo "linha 1" > app.txt
git add .
git commit -m "inicial"

echo "linha 2" >> app.txt

git diff
```

Saída:

```
+ linha 2
```

Depois:

```
git add .
git diff --staged
```

🧠 Mentalidade profissional

Você não usa `git diff` só pra ver mudança.

Você usa para:

- revisar código
- evitar erro
- entender impacto
- debugar

⚠ Erros comuns

❌ não usar diff

→ commita coisa errada

❌ confiar só no “olhômetro”

→ bug escondido

🎯 Regra de ouro

Sempre use `git diff` antes de commit

🚀 Nível engenharia

Quem domina `diff`:

- entende o que está mudando
- controla qualidade
- evita bugs silenciosos

🔥 Se quiser aprofundar mais

Posso te ensinar:

- como ler diff como um profissional
- como usar diff em projetos grandes

- ferramentas visuais (VS Code, etc.)